

Operating Systems: Lecture and Lab Notes

Prof. Mehdi Pirahandeh

Contents

1	Week 11 Lecture: Main Memory in Operating Systems	2
1.1	Objectives	2
1.2	Memory Hardware and Access	2
1.2.1	Memory Protection	2
1.3	Address Binding	2
1.4	Code Example: Using Base and Limit Registers	3
1.5	Paging and Fragmentation	3
1.6	Logical vs Physical Address Space	3
1.7	Memory-Management Unit (MMU)	3
1.8	Paging and Segmentation	4
1.9	Dynamic Loading and Linking	4
1.9.1	Advantages	4
1.10	Fragmentation	4
1.11	Translation Lookaside Buffer (TLB)	4
1.12	Swapping	4
1.13	Intel Architecture	5
1.14	Summary and Comparison	5
2	Week 11 Lab: Accessing Linux File Systems	5
2.1	Lab Objectives	5
2.2	Introduction to Linux File Systems	5
2.2.1	File System Hierarchy	5
2.3	Accessing Linux File Systems	6
2.3.1	File Permissions	6
2.4	Mounting and Unmounting File Systems	6
2.5	Practical Example of File System Mounting	6
2.5.1	Mount Command	7
2.5.2	Unmount Command	7
2.6	File System Management Commands	7
2.6.1	Checking Disk Space	7
2.6.2	File System Consistency Check	7
2.7	Comparison of Key File System Commands	7
2.8	Examples and Practical Use Cases	8
2.8.1	Mounting a USB Drive	8
2.8.2	Checking Disk Usage Regularly	8
2.8.3	Repairing a File System	8
2.9	Summary and Key Takeaways	8

1 Week 11 Lecture: Main Memory in Operating Systems

1.1 Objectives

- Detailed description of memory hardware organization.
- Overview of memory-management techniques.
- Exploration of the Intel Pentium architecture supporting segmentation and paging.

1.2 Memory Hardware and Access

Programs must be loaded into memory from disk for execution. The CPU accesses memory in cycles, with cache acting as an intermediary to reduce access time.

- Addresses and read requests.
- Addresses, data, and write requests.

Registers are accessed within a CPU clock cycle, whereas memory access may take multiple cycles.

```
1 int x = 10; // Loaded into register
2 int y = x + 5; // Accesses the register value
```

Listing 1: Basic Memory Access Example

1.2.1 Memory Protection

Memory protection ensures each process accesses only its allowed memory addresses. This is managed by base and limit registers, which define the logical address range for each process. Memory protection allows a process to access only its address space, using base and limit registers to define boundaries.

Feature	Description
Base Register	Holds the start of a process's address space.
Limit Register	Defines the extent of the process's address space.

1.3 Address Binding

Address binding occurs at compile time, load time, or execution time. Each binding level converts logical addresses to physical ones, critical for managing how programs use memory. Address binding refers to the process of associating instructions and data with specific memory locations. This can happen at three stages:

- **Compile time** - Fixed memory addresses at compile time.
- **Load time** - Adjusted when the program is loaded.
- **Execution time** - Adjusted dynamically, usually with a Memory Management Unit (MMU).

Table 2: Address Binding Comparison

Binding Time	Description
Compile Time	Converts symbolic addresses to relocatable addresses
Load Time	Converts relocatable addresses to physical addresses
Execution Time	Maps logical addresses dynamically at runtime

1.4 Code Example: Using Base and Limit Registers

```
1 int main() {
2     int base = 1000;
3     int limit = 5000;
4     if (address >= base && address < base + limit) {
5         // Access granted
6     } else {
7         // Access denied
8     }
9 }
```

Listing 2: Base and Limit Register Usage in C

1.5 Paging and Fragmentation

Paging divides memory into fixed-sized pages, enabling efficient memory use. Fragmentation types include internal (within pages) and external (between allocated spaces).

Table 3: Comparison of Paging and Segmentation

Concept	Paging	Segmentation
Division	Fixed-sized pages	Variable-sized segments
Fragmentation	Internal	External

1.6 Logical vs Physical Address Space

Logical and physical addresses are separate concepts in memory management. Logical addresses are generated by the CPU, while physical addresses represent actual memory locations.

Concept	Description
Logical Address	Address generated by the CPU, visible to the user.
Physical Address	Real address in memory hardware.

1.7 Memory-Management Unit (MMU)

The MMU dynamically maps virtual addresses to physical addresses. It uses techniques like relocation registers, which help convert logical addresses to physical addresses during execution.

1.8 Paging and Segmentation

Paging divides memory into fixed-size blocks (pages), whereas segmentation divides it based on logical units, such as functions.

```
1 Page number (p) + offset (d) => Physical Address
```

Listing 3: Paging Address Calculation

Concept	Description
Paging	Divides memory into fixed-size blocks. Efficient but can cause internal fragmentation.
Segmentation	Divides memory into logical units, e.g., code segments.

1.9 Dynamic Loading and Linking

Dynamic loading defers loading routines until needed, while dynamic linking allows routines to be linked at runtime.

1.9.1 Advantages

- Reduces memory usage by loading only necessary code.
- Improves flexibility by linking modules at runtime.

1.10 Fragmentation

Fragmentation occurs when memory allocation leaves small gaps. Two types exist:

- **Internal** - Unused space within an allocated region.
- **External** - Small free spaces outside allocated memory, leading to inefficiency.

1.11 Translation Lookaside Buffer (TLB)

The TLB speeds up address translation by caching page table entries. Effective Access Time (EAT) depends on the TLB hit ratio.

```
1 EAT = (hit ratio * memory access time) + (miss ratio * two-memory  
    access time)
```

Listing 4: Effective Access Time Calculation

1.12 Swapping

Swapping involves moving processes in and out of memory to manage limited space, especially in priority-based scheduling.

1.13 Intel Architecture

Intel CPUs like IA-32 support both segmentation and paging:

- **Segmentation** - Logical division of memory.
- **Paging** - Enables memory to be broken into blocks.

1.14 Summary and Comparison

Concept	Segmentation	Paging
Memory Division	Logical units	Fixed-size blocks
Fragmentation	External	Internal
Hardware Support	MMU required	TLB often used

2 Week 11 Lab: Accessing Linux File Systems

2.1 Lab Objectives

- Understand the structure and operations of Linux file systems.
- Learn how to mount and unmount file systems.
- Explore commands for file system management and manipulation.

2.2 Introduction to Linux File Systems

Linux file systems provide a structured way to store, organize, and manage data. Each file system is organized hierarchically, allowing users and programs to access files through a directory path. Linux file systems are accessed through system commands like `ls`, `cd`, and `pwd`, which list directories, change directories, and print working directories, respectively.

```
1 # List files
2 ls
3
4 # Change directory
5 cd /home/user
6
7 # Print current directory
8 pwd
```

Listing 5: Basic Linux Commands

2.2.1 File System Hierarchy

The root directory (`/`) is the base of the Linux file system hierarchy. Each directory and file can be accessed through its absolute path, starting from the root.

Directory	Description
<code>/bin</code>	Essential command binaries (e.g., <code>ls</code> , <code>cp</code>).
<code>/home</code>	User home directories.

/var	Variable files, including logs and system data.
/dev	Device files.

2.3 Accessing Linux File Systems

The ‘ls’ command is a fundamental tool for listing files and directories. Options like ‘-l’ provide detailed information, while ‘-a’ reveals hidden files.

```
1 # List all files in the current directory with details
2 ls -la
```

Listing 6: Using the ls command

2.3.1 File Permissions

Linux files have permissions associated with three entities: owner, group, and others. Permissions include read (‘r’), write (‘w’), and execute (‘x’).

```
1 # Add execute permission for the user
2 chmod u+x filename
```

Listing 7: Changing File Permissions

Symbol	Meaning
r	Read permission
w	Write permission
x	Execute permission

2.4 Mounting and Unmounting File Systems

Mounting integrates external storage into the Linux directory tree, while unmounting safely removes it. Mounting attaches file systems to directories, making them accessible. Unmounting detaches them, securing data integrity.

Table 9: Commands for Mounting and Unmounting

Command	Purpose	Example
mount	Mounts a file system	mount /dev/sda1 /mnt
umount	Unmounts a file system	umount /mnt

2.5 Practical Example of File System Mounting

Consider mounting an external USB drive:

```
1 # Check available disks
2 lsblk
3
4 # Mount USB drive
5 sudo mount /dev/sdb1 /media/usb
6
```

```

7 # Access files
8 cd /media/usb

```

Listing 8: Example of Mounting a USB Drive

2.5.1 Mount Command

The ‘mount’ command is used to attach a file system to a directory.

```

1 # Mount a USB drive
2 sudo mount /dev/sdb1 /mnt/usb

```

Listing 9: Mounting a File System

2.5.2 Unmount Command

The ‘umount’ command detaches the mounted file system, ensuring no data is lost.

```

1 # Unmount a USB drive
2 sudo umount /mnt/usb

```

Listing 10: Unmounting a File System

Command	Description
mount	Mount a file system to a directory.
umount	Unmount a file system from the directory tree.

2.6 File System Management Commands

2.6.1 Checking Disk Space

Use the ‘df’ command to view the disk space usage of file systems.

```

1 # Display disk space in human-readable format
2 df -h

```

Listing 11: Checking Disk Space Usage

2.6.2 File System Consistency Check

The ‘fsck’ command verifies the integrity of file systems, detecting and repairing issues.

```

1 # Check and repair /dev/sdb1
2 sudo fsck /dev/sdb1

```

Listing 12: File System Check

2.7 Comparison of Key File System Commands

Command	Usage	Example
ls	Lists files in a directory	ls -la

chmod	Changes file permissions	<code>chmod u+x file</code>
mount	Attaches a file system	<code>mount /dev/sdb1 /mnt</code>
umount	Detaches a file system	<code>umount /mnt</code>
df	Displays disk space usage	<code>df -h</code>
fsck	Checks file system integrity	<code>fsck /dev/sdb1</code>

2.8 Examples and Practical Use Cases

2.8.1 Mounting a USB Drive

To access files on a USB drive, mount it using the ‘mount’ command:

```
1 # Assuming the USB is recognized as /dev/sdb1
2 sudo mount /dev/sdb1 /mnt/usb
```

Listing 13: Mounting a USB Drive

This allows users to access files in ‘/mnt/usb’.

2.8.2 Checking Disk Usage Regularly

Administrators can use ‘df -h’ to monitor disk space and ensure enough space remains available for system operations.

2.8.3 Repairing a File System

If a file system becomes corrupted, running ‘fsck’ can help restore it:

```
1 sudo fsck /dev/sdb1
```

Listing 14: Repairing a Corrupted File System

This command repairs minor issues and flags any serious errors for further investigation.

2.9 Summary and Key Takeaways

This lab introduced fundamental Linux file system commands, focusing on mounting, unmounting, and file system checks. Proper usage of these commands ensures data integrity, efficient storage management, and smooth system operations.

Conclusion

This lecture and lab guide has presented foundational concepts in main memory management and practical usage of Linux file systems. Mastery of these concepts will support deeper OS studies and practical system management.